Information Theoretic Methods in Generative AI

Assignment 2: A Comparison Between GAN, VAE and Normalzing FLows

Released:	September 23, 2026
Due Date:	October 7, 2026
Weight:	20% of course grade

Instructions: This is a coding assignment implementing GAN variants. You are encouraged to use an AI coding assistant (e.g., Claude Code) as you would in a real research/engineering setting — but you are responsible for understanding every line of the code you submit, and should be prepared to explain your design choices. Each part should be submitted as a notebook (or script) plus a brief write-up of your results.

Part 0: Preparation — GPU Environment, Coding Agent [10 pts]

GPU Environment and Coding Agent

You will need a PyTorch environment with GPU access to complete this assignment. Two options:

- **Course GPU server (NAIRR-provisioned, Koa, Google Cloud):**
  1. We have provided you with Google education credits, which should be enough for you to finish this homework.
  2. We encourage you to use remote virtual machine SSH in: `ssh exouser@149.165.147.209`. (login password and user name are different across platform and user)
  3. **(Optional)** Install a coding agent on the server (e.g., Claude Code) to help with the rest of the assignment. We won't walk through the installation steps here — part of the exercise is asking an AI assistant itself how to install and set it up. This is a realistic taste of the kind of environment setup you'll repeatedly run into in practice.
- **Your own machine, or Google Colab:** if you have a GPU (e.g., a laptop/desktop with an NVIDIA GPU), a local PyTorch install is fine. Otherwise, Google Colab's free GPU tier is a reasonable fallback for an assignment this size (MNIST DCGAN training is not compute-heavy).

Whichever route you take, you're encouraged to use an AI coding agent to help debug environment/setup issues (CUDA mismatches, driver errors, etc.) — this is exactly the kind of friction agents are good at cutting through, and it will save you time for the actual modeling work in Parts 1+.

PyTorch Setup, Dataset Preparation, and a Simple Classifier Network

In this part we hope you can get familiar with agent coding and deep neural network training. We are going to train a neural network for classification on MNIST, and the output of its final hidden layer (before the classification head) will be used as **features to compute the FID score** in later parts of this homework.

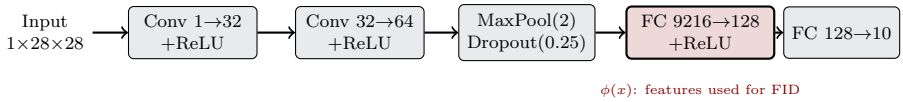
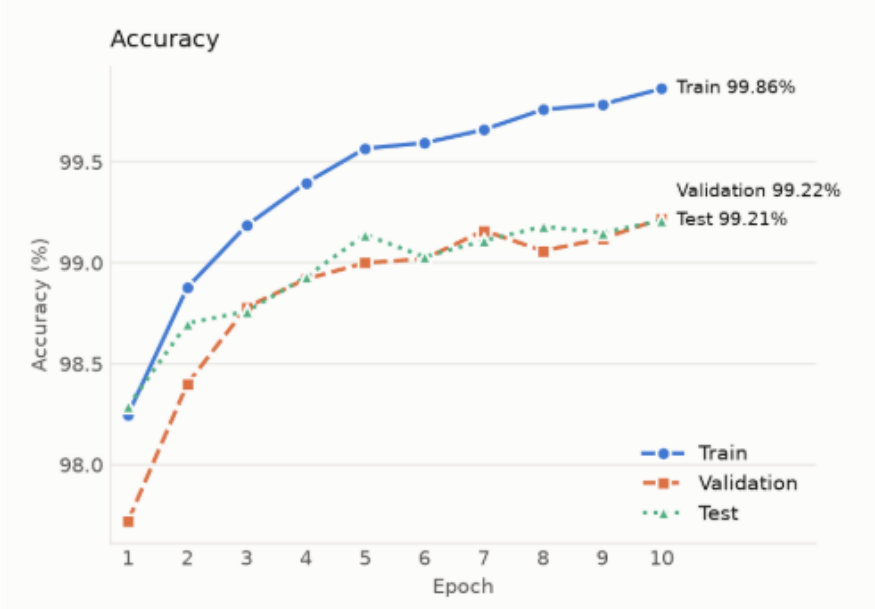


Figure 1: Illustration of Classifier

Actually, you can finish this part directly by letting your coding agent read and run through `Setup_GAN.md`'s **Task 0: Train an MNIST Classifier (for FID Features)** section, which specifies this exact architecture, training settings, and logging requirements. Train for **30 epochs**, logging training, validation, and test loss/accuracy at the end of every epoch.



Example accuracy curve (train/validation/test vs. epoch) — your plot should have this general shape, though your own numbers, epoch count, and gap between the three curves will differ.

Deliverables

1. Plot 2 figures: one showing training loss, validation loss, and test loss vs. epoch, and one showing training accuracy, validation accuracy, and test accuracy vs. epoch (see the example above for the general shape of the accuracy plot). [5 pts]

2. Discuss your findings on the differences between training, validation, and test accuracy. Then pick one saved checkpoint to keep for future use in later parts of this homework (i.e., the one whose features $\phi(\cdot)$ you'll use to compute FID) — state which epoch's checkpoint you picked and why. [5 pts]

Part 1: DCGAN [30 pts]

Give your coding agent (e.g., Claude Code) the file `Setup_GAN.md` and have it read through the instructions there, then follow along to set up the environment, prepare the MNIST dataset, and implement and train the DCGAN specified in that document. The model architecture, training settings, and checkpoint/logging requirements are all specified there — treat it as the spec for this part.

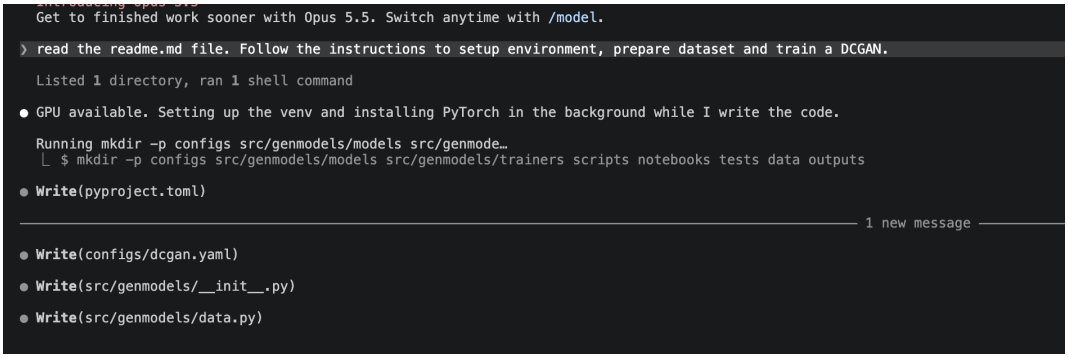


Figure 2: An illustration of a coding agent doing the implementation.

Deliverables

1. After training, store a figure consisting of at least 10 images that your GAN generated. [5 pts]

2. Understanding your code, take a photo of your discriminator loss and submit it in your report. [5 pts]

3. Evaluation: FID measures how close the distribution of generated images is to the distribution of real images, using features from a pretrained network $\phi(\cdot)$. Here we use the last layer of the classifier before the classifier as features, as illustrated in Fig. 1. Modeling $\phi(x)$ as Gaussian, let μ_r, Σ_r and μ_g, Σ_g be the mean and covariance of the features over real and generated images, respectively. Then:

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{Tr}\Big(\Sigma_r + \Sigma_g - 2\left(\Sigma_r \Sigma_g\right)^{1/2}\Big)$$

   Assuming you are given 100 samples $\{z_1, \cdots, z_{100}\}$, and the we have obtained a generator $G_\theta(\cdots) : \mathcal{Z} \mapsto \mathcal{X}$. $\phi(\cdot) : \mathcal{X} \mapsto \mathbb{R}^1 28$ feature mapping of the classifier. Write out the formula about computing μ_g and Σ_g using $\{z_1, \cdots, z_{100}\}, G_\theta, \phi$. [5 pts]

4. Increasing the learning rate of the discriminator by 20times. Then generate some digits as Fig. 2. Tell their difference, and explain the reason. [5 pts]

5. Try fine-tune the learning rate (e.g.,), train for longer epochs, report the FID scores computed by 10000 images, tell me your findings. [10pts]

   Hint: Lower FID means the generated distribution is closer to the real one.using the FID formula above, compute the FID score of your trained model

Part 2: Understand and Implement Wasserstein GAN [25pts]

In class, we understand that the training objective of GAN is:

$$\min_{G_\theta} \max_{D_\phi} \mathbb{E}_{x\sim p_{data}}[\log D_\phi(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D_\phi(G_\theta(z)))] \tag{1}$$

where p_{data} is the data distribution, p_z is the prior over the latent code z, G_θ is the generator, and D_ϕ is the discriminator.

Deliverables

1. Let p, q be two distributions. Let $M(x) := \frac{1}{2}(p(x) + q(x))$. Then the Jensen-Shannon Divergence is defined:

$$JSD(p,q) = \frac{1}{2}\left(D_{KL}(p,M) + D_{KL}(q,M)\right) = \frac{1}{2}\left(\mathbb{E}_p\left[\log\frac{p(x)}{M(x)}\right] + \mathbb{E}_q\left[\log\frac{q(x)}{M(x)}\right]\right) \tag{2}$$

   Explain why training the GAN is actually minimizing the Jensen-Shannon divergence.[10pts]

2. The Wasserstein GAN replaces this objective with the Kantorovich–Rubinstein dual form of the Wasserstein-1 distance:

$$\min_{G_\theta} \max_{D_\phi:\, \|D_\phi\|_L \le 1} \mathbb{E}_{x\sim p_{data}}\big[D_\phi(x)\big] - \mathbb{E}_{z\sim p_z}\big[D_\phi(G_\theta(z))\big] \tag{3}$$

   where $\|D_\phi\|_L \le 1$ restricts D_ϕ to be 1-Lipschitz. Compare this to the vanilla GAN objective in Equation (1), and discuss what needs to change to train it (e.g., what happens to the sigmoid/log on D_ϕ's output, and how you would enforce $\|D_\phi\|_L \le 1$ in practice). [10pts]

3. Implement your change, fine-tuing the paramters (learning rate, epochs), report the best FID scoures you get and your observations, findings in this round. [5pts]

Part 3: Train a VAE Model [35 pts]

In this part, we can keep the same Generator architecture and use it to train the VAE models. It will enable a fair comparison of how different training algorithm affects the generation quality.

Let E_ϕ be the encoder mapping x to a Gaussian posterior $q_\phi(z \mid x) = \mathcal{N}\big(z; \mu_\phi(x), \sigma^2_\phi(x)\big)$, and let G_θ be the decoder (reusing your generator architecture) parameterizing $p_\theta(x \mid z)$. With prior $p(z) = \mathcal{N}(0, I)$, the VAE training loss — a β-weighted ELBO — is:

$$\mathcal{L}_{\text{VAE}}(\theta,\phi) = \mathbb{E}_{x\sim p_{data}}\Big[\underbrace{-\mathbb{E}_{z\sim q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big]}_{\text{reconstruction}} + \beta\, \underbrace{D_{KL}\big(q_\phi(z \mid x)\,\|\,p(z)\big)}_{\text{regularization}}\Big] \tag{4}$$

where $\beta \ge 0$ trades off reconstruction fidelity against how closely the latent posterior is pushed toward the prior ($\beta = 1$ recovers the standard VAE/ELBO).

1. Design an encoder by reverting the layer of your generator. Write out your choice of encoder architecture [5 pts]

2. Use your encoder architecture to train a VAE models. Fine-tuning the learning rate and training for longer epochs. Report the loss curve. Then, select a model checkpoint to evaluate the FID score. [10 pts]

3. Understand the latent space. Generate an set of image, where we keep $z_{11:100}$ the same and modify $z_{0:10}$. Report your findings, and talk about how this finding can be used. [5pts]

4. Try using $\beta = 0.1, 10$ in the training, and try annealing strategy ($\beta = 0.01$ at the first epoch, then gradually increase to 1 in 10 epochs.) Compare the image generated from the different recepies. Write your findings. [10pts]

5. Report the FID score of your best VAE models, and compare it with GANs and Wasserstein-GANs. [5pts]